How work grows: big O

Programming · Level 4 · Lesson 1



In this lesson you will

- Count the steps an algorithm takes as the input grows.
- Classify code as O(1), O(log n), O(n), O(n log n) or $O(n^2)$.
- Replace an $O(n^2)$ method with a faster one.

Key ideas

- Count steps, not seconds.
- Double n: O(n) doubles, $O(n^2)$ quadruples.
- A better method beats a faster phone.

Activity: Every pair, or one pass?

1. Warm-up (3 min): Guess how long a duplicate check takes for a million.
2. Count (7 min): Count every-pair comparisons for 4, 8 and 16 items.
3. Improve (7 min): Rewrite it with a set; count again.
4. Explain (3 min): State both in big O.

Counting on paper is enough: the pattern appears by 16 items.

Let's discuss

1. Why do computer scientists count steps instead of seconds?
2. Where have you seen a hidden loop?
3. When is $O(n^2)$ good enough?

Exit ticket

- Write a loop that is O(n), and one that is $O(n^2)$, for the same list.